

Case Técnico SRE DevOps

Lagunitas / HEINEKEN

Autor: Laerte Gomes Filho

Data: 06/04/2026

#	Entregável	Seção
1	Diagnóstico: o que falhou no pipeline e por quê	Seção 1
2	YAML refatorado com gate de aprovação e smoke tests (comentado)	Seção 2
3	Proposta de observabilidade do deploy (com steps de log no YAML)	Seção 3
4	Estratégia de rollback: o que automatizar e o que é manual	Seção 4
5	Questões de aprofundamento (DABs, Key Vault, alerta proativo)	Seção 5
6	Evidências do pipeline em execução (GitHub Actions)	Seção 6
7	Conclusão	Seção 7

1. Diagnóstico — O que falhou no pipeline e por quê

1.1 Causas raiz do incidente

Causas técnicas:

- **1. Variável DATABRICKS_HOST apontando para DEV em vez de PRD.** O token/service connection tinha permissão em DEV e PRD simultaneamente. A variável DATABRICKS_HOST foi resolvida com o valor de DEV, e o comando databricks workspace import_dir executou sem erro porque o host era válido. Não havia nenhum step que comparasse o host de destino com o ambiente esperado.
- **2. ARM Template deploy sem validação de estado funcional.** O ARM Template foi aplicado com sucesso (exit code 0), mas dois pipelines ADF ficaram inativos e as triggers não foram reativadas. O Azure DevOps só verifica o resultado da operação de deployment, não o estado funcional dos recursos pós-deploy.
- **3. Zero smoke tests pós-deploy.** Nenhum step pós-deploy verificava: workspace de destino correto, notebooks no path esperado, jobs ativos, pipelines ADF Enabled, triggers Running. O pipeline terminou "verde" com deploy no lugar errado.
- **4. Service Principal / token com escopo excessivo.** Um único token com escopo em todos os ambientes viola o princípio de least privilege. Se cada ambiente tivesse seu próprio token via Key Vault separado, o deploy no ambiente errado falharia por falta de permissão.

Causas de processo:

- **1. Ausência de gate de aprovação antes de PRD.** Qualquer merge na main ia direto para produção. Não havia revisão humana entre build e deploy PRD.
- **2. ACC não utilizado como gate formal.** O ambiente ACC existe na infraestrutura mas não fazia parte da esteira CI/CD. Não havia deploy intermediário nem validação antes de PRD.
- **3. Sem runbook ou automação de rollback.** Quando o incidente foi detectado (4 horas depois, por um stakeholder), a remediação foi 100% manual e levou 42 minutos. Não existia procedimento documentado nem automação de rollback.
- **4. Descoberta reativa pelo negócio, não pelo pipeline.** A ausência de monitoramento proativo fez com que o time de dados só soubesse do problema quando o analista de negócios abriu chamado às 18h20.

1.2 Por que o pipeline reportou sucesso com deploy errado?

O pipeline original verificava apenas:

- **Build:** py_compile nos notebooks — verifica exclusivamente sintaxe Python.
- **Deploy:** Exit code dos comandos databricks workspace import_dir e AzureResourceManagerTemplateDeployment@3. Ambos retornam 0 se a operação técnica completou, independente de destino ou estado funcional.

O conceito errado é tratar "operação sem erro" como "deploy funcional". O pipeline deveria ter verificado:

- Que o DATABRICKS_HOST resolvia para o workspace de PRD (não DEV).
- Que os notebooks existiam no path /Shared/lagunitas no workspace PRD após o import.
- Que os jobs Databricks associados estavam ativos e apontando para a versão correta.
- Que os pipelines ADF no factory de PRD estavam com status Enabled.
- Que as triggers ADF dependentes estavam com status Started.

1.3 O lint (py_compile) é suficiente como validação?

O que py_compile detecta: erros de sintaxe Python (parênteses, indentação, keywords).

O que py_compile não detecta:

Tipo de problema	Exemplo
Erros de lógica	JOIN errado, filtro invertido, coluna trocada
Imports inexistentes	Biblioteca não instalada no cluster Databricks
Referências a objetos inexistentes	Tabela/schema/catalog que não existe no ambiente
Erros de runtime PySpark	Schema mismatch, null handling, type casting
Performance	Full table scans, cross joins acidentais
Variáveis de ambiente	Variável ausente ou com valor incorreto no ambiente de destino

Recomendação: adicionar ruff ou flake8 (linting avançado), mypy (type checking), e pytest + chispa (testes unitários PySpark contra dados mock).

1.4 Se pudesse implementar apenas uma melhoria agora

Gate de aprovação manual antes de PRD com Environment protegido.

Das quatro falhas (sem gate, sem smoke test, sem observabilidade, sem rollback), **o gate é a única que previne o incidente**. As demais melhoram detecção e recuperação, mas não impedem o deploy errado de acontecer.

Esforço de implementação: mínimo. Basta criar um Environment "lagunitas-prd" no Azure DevOps com required approvers e referenciá-lo no stage. Zero código, impacto imediato.

Quem deveria aprovar: Tech Lead de Dados (valida o que está sendo deployado) + SRE de plantão (valida a saúde da plataforma). O aprovador precisa ver: commit SHA, diff das mudanças, quem fez o merge e resultado dos smoke tests de ACC.

2. Pipeline Refatorado (azure-pipelines.yml)

Abaixo está o YAML completo refatorado e comentado. O arquivo azure-pipelines.yml no repositório é idêntico a este conteúdo.

2.1 Trigger e variáveis globais

```
# Trigger: dispara automaticamente em merge na main
trigger:
  branches:
    include:
      - main

# Variáveis globais - commit e autor capturados para rastreabilidade
variables:
  - name: COMMIT_SHA
    value: $(Build.SourceVersion)
  - name: MERGED_BY
    value: $(Build.RequestedFor)
```

Mudança vs original: COMMIT_SHA e MERGED_BY são capturados para o log estruturado de deploy.

2.2 Stage: Build & Validate

```
# STAGE 1: Validação de artefatos antes de qualquer deploy
- stage: Build
  displayName: 'Build & Validate Artifacts'
  jobs:
    - job: BuildJob
      pool:
        vmImage: 'ubuntu-latest'
      steps:
        - checkout: self

        # Lint 1: py_compile (mantido do original - sintaxe básica)
        - script: |
            find ./notebooks -name '*.py' | xargs python -m py_compile
            displayName: 'Lint notebooks (py_compile)'

        # Lint 2: ruff para problemas além de sintaxe (NOVO)
        - script: |
            pip install ruff
            ruff check ./notebooks --select E,W,F --ignore E501
            displayName: 'Lint notebooks (ruff)'

        # Validação de ARM Templates do ADF (NOVO)
        - script: |
            python -m json.tool ./adf/ARMTemplateForFactory.json > /dev/null
            python -m json.tool ./adf/ARMTemplateParametersForFactory.json > /dev/null
            displayName: 'Validate ADF ARM templates'

        # Publicação de artefatos para os stages seguintes
        - publish: $(Build.ArtifactStagingDirectory)
          artifact: drop
```

Mudanças vs original: adicionado ruff (linting avançado) e validação de ARM templates JSON.

2.3 Stage: Deploy DEV (automático)

```
# STAGE 2: Deploy automático em DEV (sem gate)
- stage: DeployDEV
  dependsOn: Build
  variables:
    # Variable Group vinculado ao Key Vault de DEV
    # Contém DATABRICKS_HOST e DATABRICKS_TOKEN de DEV apenas
    - group: lagunitas-dev-vars
  jobs:
    - deployment: DeployDevJob
      # Environment sem aprovação - deploy automático
      environment: 'lagunitas-dev'
      strategy:
        runOnce:
          deploy:
            steps:
              - download: current
                artifact: drop
                # Deploy Databricks notebooks em DEV
              - task: AzureCLI@2
                displayName: 'Deploy Databricks Notebooks (DEV)'
                inputs:
                  azureSubscription: 'lagunitas-dev-service-connection'
                  scriptType: bash
                  inlineScript: |
                    databricks workspace import_dir ./notebooks /Shared/lagunitas \
                      --host $(DATABRICKS_HOST) --token $(DATABRICKS_TOKEN) --overwrite
                # Deploy ADF ARM Template em DEV
              - task: AzureResourceManagerTemplateDeployment@3
                displayName: 'Deploy ADF ARM Template (DEV)'
                inputs:
                  deploymentScope: 'Resource Group'
                  azureResourceManagerConnection: 'lagunitas-dev-service-connection'
                  resourceGroupName: $(ADF_RESOURCE_GROUP)
                  location: 'Brazil South'
                  csmFile: '$(Pipeline.Workspace)/drop/adf/ARMTemplateForFactory.json'
                  csmParametersFile:
                    '$(Pipeline.Workspace)/drop/adf/ARMTemplateParametersForFactory.json'
                  deploymentMode: 'Incremental'
```

Mudança-chave: Variable Group separado (lagunitas-dev-vars) com credenciais exclusivas de DEV via Key Vault.

2.4 Stage: Deploy ACC (automático + smoke tests)

```
# STAGE 3: Deploy ACC com smoke tests de validação
# ACC funciona como gate de qualidade antes da aprovação para PRD
- stage: DeployACC
  dependsOn: DeployDEV
  variables:
    - group: lagunitas-acc-vars
  jobs:
    - deployment: DeployAccJob
      environment: 'lagunitas-acc'
      strategy:
        runOnce:
          deploy:
            steps:
              # ... (deploy idêntico ao DEV, com service connection de ACC)

              # SMOKE TESTS em ACC (NOVOS - validam mecânica antes de PRD)
              - script: |
                  bash scripts/smoke-test-databricks.sh \
                    "$(DATABRICKS_HOST)" "$(DATABRICKS_TOKEN)" "/Shared/lagunitas" "ACC"
                  displayName: 'Smoke Test - Databricks (ACC)'

              - script: |
                  bash scripts/smoke-test-adf.sh \
                    "$(ADF_RESOURCE_GROUP)" "$(ADF_FACTORY_NAME)" "ACC"
                  displayName: 'Smoke Test - ADF (ACC)'
```

Novidade: ACC agora faz parte da esteira com smoke tests. O aprovador de PRD verá os resultados de ACC antes de aprovar.

2.5 Stage: Deploy PRD (gate de aprovação obrigatório)

```
# STAGE 4: Deploy PRD com aprovação manual
# -----
# CONFIGURAÇÃO DO ENVIRONMENT 'lagunitas-prd' NO AZURE DEVOPS:
# 1. Project Settings > Environments > New: 'lagunitas-prd'
# 2. Approvals and checks > Add: Required approvers
# 3. Aprovadores: Tech Lead de Dados + SRE de plantão
# 4. Timeout: 4 horas
# 5. Exclusive lock: apenas um deploy por vez
#
# INFORMAÇÃO VISÍVEL AO APROVADOR:
# - Commit SHA e mensagem do merge
# - Quem fez o merge (MERGED_BY)
# - Resultado dos smoke tests de ACC (stage anterior)
# - Link para o diff no Azure Repos
# -----
- stage: DeployPRD
  dependsOn: DeployACC
  variables:
    - group: lagunitas-prd-vars
  jobs:
    - deployment: DeployPrdJob
      # Este environment EXIGE aprovação manual
      environment: 'lagunitas-prd'
      strategy:
        runOnce:
          deploy:
            steps:
              - download: current
                artifact: drop

              # SNAPSHOT pré-deploy do ADF para rollback (NOVO)
              - task: AzureCLI@2
                displayName: 'Snapshot ADF state (pre-deploy)'
                inputs:
                  azureSubscription: 'lagunitas-prd-service-connection'
                  scriptType: bash
                  inlineScript: |
                    az datafactory pipeline list \
                      --resource-group $(ADF_RESOURCE_GROUP) \
                      --factory-name $(ADF_FACTORY_NAME) \
                      --output json > $(Pipeline.Workspace)/adf-snapshot.json
              - publish: $(Pipeline.Workspace)/adf-snapshot.json
                artifact: adf-rollback-snapshot

              # VALIDAÇÃO DE SEGURANÇA: host DEVE conter 'prd' (NOVO)
              - script: |
                  if [[ "$(DATABRICKS_HOST)" != *"prd"* ]]; then
                    echo "##[error] DATABRICKS_HOST não contém 'prd'. Deploy abortado."
                    exit 1
                  fi
                displayName: 'Validate Databricks host is PRD'

              # Deploy Databricks notebooks em PRD
              - task: AzureCLI@2
                displayName: 'Deploy Databricks Notebooks (PRD)'
                inputs:
                  azureSubscription: 'lagunitas-prd-service-connection'
                  scriptType: bash
                  inlineScript: |
                    databricks workspace import_dir ./notebooks /Shared/lagunitas \
                      --host $(DATABRICKS_HOST) --token $(DATABRICKS_TOKEN) --overwrite

              # Deploy ADF ARM Template em PRD
              - task: AzureResourceManagerTemplateDeployment@3
                displayName: 'Deploy ADF ARM Template (PRD)'
                inputs:
                  deploymentMode: 'Incremental'
                  # ... (mesmos parâmetros, com service connection PRD)

              # REATIVAÇÃO de triggers ADF pós-deploy (NOVO)
              - task: AzureCLI@2
                displayName: 'Reactivate ADF triggers (PRD)'
                inputs:
```

```
inlineScript: |
  TRIGGERS=$(az datafactory trigger list \
    --resource-group $(ADF_RESOURCE_GROUP) \
    --factory-name $(ADF_FACTORY_NAME) \
    --query "[].name" -o tsv)
  for trigger in $TRIGGERS; do
    az datafactory trigger start \
      --resource-group $(ADF_RESOURCE_GROUP) \
      --factory-name $(ADF_FACTORY_NAME) \
      --name "$trigger"
  done
```

Mudanças vs original: (1) snapshot pré-deploy para rollback, (2) validação de host, (3) reativação automática de triggers, (4) gate de aprovação via Environment.

3. Observabilidade do Deploy

3.1 Destino dos logs: Log Analytics via Azure Monitor

Escolha: Log Analytics Workspace.

Critério	Justificativa
Integração nativa	Log Analytics é o destino padrão para logs no ecossistema Azure que a Lagunitas já utiliza.
Troubleshooting rápido	Queries KQL permitem investigar deploys em minutos. O RCA que levou 40 min levaria 5 min.
Alertas proativos	Integra com Azure Monitor Alerts para regras como "deploy sem execução ADF em 2h".
Dashboards	Workbooks permitem histórico visual de deploys para o time e liderança.
Custo	Cobrança por GB ingerido. O volume de logs de deploy é muito baixo, com custo praticamente irrelevante (centavos por mês).

3.2 Stage de Smoke Tests + Log Estruturado (YAML)

Este stage roda após o deploy PRD e executa dois blocos: smoke tests e emissão do log.

```
# STAGE 5: Smoke Tests + Log Estruturado
- stage: SmokeTestsPRD
  dependsOn: DeployPRD
  variables:
    - group: lagunitas-prd-vars
  jobs:
    - job: SmokeAndLog
      steps:

        # --- SMOKE TEST DATABRICKS ---
        # Verifica: (1) host é PRD, (2) notebooks existem no path,
        # (3) jobs Databricks associados estão ativos
        - script: |
            bash scripts/smoke-test-databricks.sh \
              "$(DATABRICKS_HOST)" "$(DATABRICKS_TOKEN)" "/Shared/lagunitas" "PRD"
            displayName: 'Smoke Test - Databricks (PRD)'
            name: smokeDbx

        # --- SMOKE TEST ADF ---
        # Verifica: (1) factory acessível, (2) pipelines Enabled,
        # (3) triggers Started, (4) factory name corresponde a PRD
        - script: |
            bash scripts/smoke-test-adf.sh \
              "$(ADF_RESOURCE_GROUP)" "$(ADF_FACTORY_NAME)" "PRD"
            displayName: 'Smoke Test - ADF (PRD)'
            name: smokeAdf

        # --- LOG ESTRUTURADO ---
        # Emite JSON com todos os dados do deploy para Log Analytics
        # via HTTP Data Collector API
        - script: |
            bash scripts/emit-deploy-log.sh \
              "PRD" "$(COMMIT_SHA)" "$(MERGED_BY)" \
              "$(DATABRICKS_HOST)" "$(ADF_FACTORY_NAME)" \
              "$(LOG_ANALYTICS_WORKSPACE_ID)" "$(LOG_ANALYTICS_SHARED_KEY)"
            displayName: 'Emit deploy log to Log Analytics'
```


3.3 Estrutura do log JSON emitido

```
{ "timestamp": "2026-04-02T14:42:00Z",
  "pipeline_run_id": "20260402.7",
  "commit_sha": "alb2c3d",
  "commit_message": "fix: silver notebooks join logic",
  "merged_by": "joao.silva@lagunitas.com",
  "approved_by": "maria.santos@lagunitas.com",
  "environment": "PRD",
  "deploy_components": {
    "databricks": { "host": "https://adb-prd...", "path": "/Shared/lagunitas",
      "notebooks": [ "silver_vendas.py", "silver_distribuicao.py"], "status": "success" },
    "adf": { "factory": "adf-lagunitas-prd",
      "pipelines": [ "pl_silver_vendas", "pl_silver_distribuicao"], "status": "success" }
  },
  "smoke_tests": {
    "databricks_workspace_validation": "PASS",
    "databricks_notebooks_exist": "PASS",
    "databricks_jobs_active": "PASS",
    "adf_pipelines_enabled": "PASS",
    "adf_triggers_active": "PASS" },
  "overall_status": "SUCCESS",
  "duration_seconds": 187 }
```

3.4 Query KQL para responder: "o deploy das 14h42 deployou o quê, onde, e estava ok?"

```
DeployLogs_CL
| where TimeGenerated between (datetime(2026-04-01 14:00) .. datetime(2026-04-01 15:00))
| project TimeGenerated, commit_sha_s, environment_s, merged_by_s, approved_by_s,
  deploy_components_databricks_notebooks_s, deploy_components_adf_pipelines_s,
  smoke_tests_overall_s, overall_status_s
```

Com essa única query, qualquer pessoa do time responde em segundos o que foi deployado, onde, por quem e se tudo estava funcional depois.

3.5 Query KQL operacionais

A query apresentada na seção 3.4 foi elaborada especificamente para ilustrar o RCA do incidente de terça-feira, com horário fixo. No entanto, para que o time consiga operar com autonomia no dia a dia, é fundamental dispor de um conjunto de queries prontas para os cenários mais recorrentes de investigação e acompanhamento.

Abaixo estão as queries recomendadas, organizadas por caso de uso.

Consulta rápida — último deploy em PRD:

Responde à pergunta "o que foi deployado por último em produção e deu tudo certo?"

```
DeployLogs_CL
| where environment_s == "PRD"
| top 1 by TimeGenerated desc
| project TimeGenerated, commit_sha_s, merged_by_s, approved_by_s,
  overall_status_s, smoke_tests_s
```

Histórico recente — últimos 10 deploys:

Visão consolidada para revisão rápida da esteira.

```
DeployLogs_CL
| where environment_s == "PRD"
| top 10 by TimeGenerated desc
| project TimeGenerated, commit_sha_s, merged_by_s, overall_status_s
```

Deteção de falhas — deploys com problema nas últimas 24 horas:

Útil para verificação de início de turno ou após um período sem acompanhamento.

```
DeployLogs_CL
| where TimeGenerated > ago(24h)
| where overall_status_s != "SUCCESS"
| project TimeGenerated, environment_s, commit_sha_s, merged_by_s,
    smoke_tests_s, overall_status_s
```

Investigação pontual — busca por commit SHA:

Quando o time precisa rastrear exatamente o que aconteceu com um deploy específico.

```
DeployLogs_CL
| where commit_sha_s startswith "a1b2c3d"
| project TimeGenerated, environment_s, merged_by_s, approved_by_s,
    smoke_tests_s, overall_status_s
```

Visão gerencial — frequência de deploys e falhas por dia (últimos 30 dias):

Permite acompanhar a saúde da esteira ao longo do tempo e identificar tendências.

```
DeployLogs_CL
| where TimeGenerated > ago(30d)
| summarize deploys = count(), falhas = countif(overall_status_s != "SUCCESS")
    by bin(TimeGenerated, 1d), environment_s
| order by TimeGenerated desc
```

4. Estratégia de Rollback

4.1 Databricks Notebooks

Procedimento automatizado (via pipeline):

- O repositório Git é a fonte de verdade. Cada commit é um snapshot completo dos notebooks.
- Para rollback: o pipeline recebe o SHA anterior como parâmetro e executa `git checkout <sha> -- ./notebooks` seguido de `databricks workspace import_dir` com `--overwrite`.
- Após o re-import, os mesmos smoke tests são executados para validar que o rollback foi efetivo.
- Tempo estimado: aproximadamente 3 minutos (checkout + import + validação).

Comando de rollback:

```
# Pipeline de rollback parametrizado
parameters:
  - name: ROLLBACK_SHA
    type: string
    displayName: 'Commit SHA para rollback'

steps:
  - checkout: self
  - script: |
    git checkout $(ROLLBACK_SHA) -- ./notebooks
    databricks workspace import_dir ./notebooks /Shared/lagunitas \
      --host $(DATABRICKS_HOST) --token $(DATABRICKS_TOKEN) --overwrite
    displayName: 'Rollback notebooks to $(ROLLBACK_SHA)'

# Smoke tests pós-rollback (obrigatórios)
- script: |
  bash scripts/smoke-test-databricks.sh \
    "$(DATABRICKS_HOST)" "$(DATABRICKS_TOKEN)" "/Shared/lagunitas" "PRD"
```

Limitações sem Asset Bundles (DABs):

- O `import_dir` sobrescreve todo o conteúdo do path. Não há versionamento nativo no workspace.
- Se um notebook foi renomeado entre versões, o rollback deixa o notebook antigo órfão no workspace (`import_dir` não deleta arquivos que não estão no source).
- Jobs Databricks que referenciam notebooks por path não são atualizados automaticamente no rollback. Se o path mudou, o job quebra.
- Não há deployment history no Databricks. A rastreabilidade depende exclusivamente do Git e dos logs de deploy no Log Analytics.

4.2 ADF ARM Templates

O que o Azure guarda por padrão:

- O Azure Resource Manager mantém histórico de deployments no resource group (últimas 800 entradas). Consultável via `az deployment group list --resource-group <rg>`.
- Porém, o deployment history salva o template aplicado, não o estado anterior dos recursos. E o modo Incremental não remove recursos adicionados.

O que o pipeline armazena explicitamente:

- Antes de cada deploy PRD, o stage faz um snapshot do estado atual do ADF (`az datafactory pipeline list + trigger list`) e publica como artefato no Azure DevOps (artifact: `adf-rollback-snapshot`).
- Esse snapshot permite reconstruir exatamente o estado anterior do factory.

Procedimento de rollback ADF:

Passo a passo:

- Baixar o artefato `adf-rollback-snapshot` da run anterior no Azure DevOps.
- Parar todas as triggers ativas: `az datafactory trigger stop --name <trigger>`.
- Re-aplicar o ARM Template da versão anterior. Se recursos foram adicionados pelo deploy que queremos reverter, usar `deploymentMode: Complete` (em vez de Incremental) para remover recursos extras.
- Reativar triggers: `az datafactory trigger start --name <trigger>` para cada trigger no snapshot.

- Executar smoke tests para validar que pipelines estão Enabled e triggers estão Started.

Comando de rollback ADF:

```
# 1. Parar triggers
az datafactory trigger stop --resource-group $RG --factory-name $FACTORY --name $TRIGGER

# 2. Re-aplicar ARM Template anterior
az deployment group create \
  --resource-group $RG \
  --template-file adf-snapshot-pre-deploy/ARMTemplateForFactory.json \
  --parameters adf-snapshot-pre-deploy/ARMTemplateParametersForFactory.json \
  --mode Complete

# 3. Reativar triggers
az datafactory trigger start --resource-group $RG --factory-name $FACTORY --name $TRIGGER

# 4. Validar
bash scripts/smoke-test-adf.sh $RG $FACTORY PRD
```

Cenários de rollback e decisão:

Cenário	Modo ARM	Ação adicional
Notebook com bug	N/A	Rollback Databricks via SHA. ADF não precisa mudar.
Pipeline ADF incorreto	Incremental	Re-apply template anterior. Triggers re-start manual.
Recurso ADF adicionado indevido	Complete	Complete remove o recurso extra. Cuidado: remove tudo que não está no template.
Ambos (notebook + ADF)	Incremental	Rollback Databricks primeiro, depois ADF. Smoke tests em ambos.
Deploy no workspace errado	N/A	Re-deploy no workspace correto. Limpar notebooks do workspace errado manualmente.

4.3 RTO (Recovery Time Objective)

RTO proposto: 15 minutos.

Etapas	Tempo	Detalhes
Deteção	~0 min	Smoke tests falham imediatamente no pipeline. Alerta automático.
Decisão go/no-go	~3 min	SRE avalia logs, decide entre fix-forward ou rollback.
Rollback Databricks	~3 min	git checkout SHA + import_dir + smoke tests.
Rollback ADF	~5 min	Stop triggers + ARM re-deploy + start triggers + smoke tests.
Validação final	~2 min	Smoke tests completos + conferir primeiro pipeline run.
Comunicação	~2 min	Notificar time de dados e stakeholders que rollback foi executado.
TOTAL	~15 min	vs 42 min do incidente + 4h de detecção (melhoria: detecção imediata, recovery 64% mais rápido)

A grande melhoria não é só no tempo de rollback (15 min vs 42 min) mas **na detecção**: com smoke tests, o incidente seria detectado em segundos, não em 4 horas pelo stakeholder.

5. Questões de Aprofundamento

5.1 Databricks Asset Bundles (DABs) vs CLI import_dir

O deploy atual utiliza databricks workspace import_dir, que é essencialmente uma cópia de arquivos para o workspace. Databricks Asset Bundles (DABs) representam uma evolução significativa nessa abordagem.

Aspecto	CLI (import_dir)	DABs
Versionamento	Nenhum no workspace	Managed deployments com histórico
Rollback	Manual (re-import SHA)	databricks bundle destroy + re-deploy
Rastreabilidade	Nenhuma nativa	Deployment metadata (quem, quando, o quê)
Configuração por ambiente	Variáveis externas	databricks.yml com targets (dev/stg/prod)
Jobs/Clusters	Gerenciados separadamente	Declarados no bundle, deployados junto
Validação	Nenhuma	databricks bundle validate pré-deploy

Como DABs resolveriam o incidente da terça-feira:

- O arquivo databricks.yml declara os targets de cada ambiente com hosts e tokens distintos. Não existe possibilidade de apontar para o workspace errado por variável mal configurada — o target é declarativo e versionado no Git.
- O comando databricks bundle validate verifica, antes do deploy, que todos os recursos referenciados existem no target de destino. Se o target fosse DEV mas o pipeline esperasse PRD, a validação falharia.
- Cada deployment cria um registro no workspace com metadata: quem deployou, quando, o quê, a partir de qual commit. Isso elimina a necessidade de log externo para rastreabilidade básica.
- O rollback com DABs é atômico: databricks bundle destroy remove tudo que o bundle criou, e um re-deploy do bundle anterior restaura o estado completo — notebooks, jobs e cluster configs juntos.

Exemplo de databricks.yml para a Lagunitas:

```
bundle:
  name: lagunitas-data-platform

targets:
  dev:
    workspace:
      host: https://adb-lagunitas-dev.azuredatabricks.net
      default: true
  acc:
    workspace:
      host: https://adb-lagunitas-acc.azuredatabricks.net
  prd:
    workspace:
      host: https://adb-lagunitas-prd.azuredatabricks.net

resources:
  jobs:
    silver_vendas:
      name: "[${bundle.target}] Silver Vendas"
      tasks:
        - task_key: transform
          notebook_task:
            notebook_path: ./notebooks/silver_vendas.py
    silver_distribuicao:
      name: "[${bundle.target}] Silver Distribuição"
      tasks:
        - task_key: transform
          notebook_task:
            notebook_path: ./notebooks/silver_distribuicao.py
```

Fluxo de deploy com DABs no pipeline:

```
steps:
- script: databricks bundle validate --target prd
  displayName: 'Validate bundle (PRD)'
- script: databricks bundle deploy --target prd
  displayName: 'Deploy bundle (PRD)'
# Rollback se necessário:
- script: |
  databricks bundle destroy --target prd --auto-approve
  git checkout $(ROLLBACK_SHA) -- .
  databricks bundle deploy --target prd
  displayName: 'Rollback bundle (PRD)'
```

O que melhora na rastreabilidade: com `import_dir`, não existe nenhum registro nativo no workspace sobre o que foi deployado, quando, por quem ou a partir de qual versão. A única rastreabilidade disponível é a que construímos externamente via logs no Log Analytics (seção 3).

Com DABs, cada deployment cria automaticamente um registro no workspace Databricks com metadata completa: timestamp, usuário, commit SHA, target de destino e lista de recursos deployados. Isso significa que, mesmo sem o log externo, qualquer pessoa com acesso ao workspace consegue consultar o histórico de deployments e identificar exatamente o que mudou e quando.

No cenário do incidente, o time teria visto imediatamente no workspace que o último deployment apontava para o target errado. O que melhora no rollback: com `import_dir`, o rollback depende de executar `git checkout` do SHA anterior e re-importar os notebooks — um processo manual que não garante consistência, pois jobs e cluster configs não são revertidos juntos. Com DABs, o rollback é atômico e gerenciado: o comando `databricks bundle destroy` remove todos os recursos criados pelo bundle (notebooks, jobs, cluster configs), e um re-deploy do bundle na versão anterior restaura o estado completo como uma unidade. Não há risco de notebooks voltarem para a versão anterior enquanto jobs continuam apontando para paths desatualizados.

Além disso, o comando `databricks bundle validate`, executado antes do deploy, verifica que todos os recursos referenciados existem no target de destino, uma validação que `import_dir` simplesmente não faz.

Recomendação: a migração de `import_dir` para DABs deveria ser o próximo passo após estabilizar a esteira com as melhorias propostas neste case. A adoção pode ser incremental, começando pelo ambiente DEV para validar o fluxo, depois ACC e por fim PRD.

5.2 Variable Groups + Key Vault para isolamento de ambiente

A causa raiz do incidente (token com acesso a DEV e PRD) é eliminada com esta arquitetura de isolamento por credencial.

Arquitetura proposta:

Ambiente	Key Vault	Secrets
DEV	kv-lagunitas-dev	DATABRICKS-TOKEN-DEV, DATABRICKS-HOST-DEV, ADF-CONN-DEV
ACC	kv-lagunitas-acc	DATABRICKS-TOKEN-ACC, DATABRICKS-HOST-ACC, ADF-CONN-ACC
PRD	kv-lagunitas-prd	DATABRICKS-TOKEN-PRD, DATABRICKS-HOST-PRD, ADF-CONN-PRD

Configuração no Azure DevOps:

- Cada Variable Group (lagunitas-dev-vars, lagunitas-acc-vars, lagunitas-prd-vars) é vinculado ao Key Vault do respectivo ambiente via "Link secrets from an Azure Key Vault".
- O pipeline referencia o Variable Group correto em cada stage. O stage de DEV usa lagunitas-dev-vars, o de PRD usa lagunitas-prd-vars.
- Os Service Principals são distintos por ambiente. O SP de DEV tem RBAC apenas no resource group de DEV; o de PRD apenas no resource group de PRD.
- Mesmo que alguém troque manualmente o valor de uma variável no pipeline, o token não tem permissão no ambiente errado. O deploy falha com 403 Forbidden — falha segura.

Como isso teria prevenido o incidente:

No cenário do incidente, a variável `DATABRICKS_HOST` apontava para DEV. Com a arquitetura de Key Vault separado, o token disponível no stage de PRD seria o `DATABRICKS-TOKEN-PRD`, que não tem permissão no workspace DEV. O comando `import_dir` falharia com erro de autenticação, e o pipeline reportaria falha (não sucesso). A falha seria detectada imediatamente, não em 4 horas.

YAML do Variable Group vinculado a Key Vault:

```
# No Azure DevOps: Library > Variable Groups > lagunitas-prd-vars
# Link secrets from Azure Key Vault: kv-lagunitas-prd
# Authorize: lagunitas-prd-service-connection
# Secrets selecionados:
#   - DATABRICKS-TOKEN-PRD → $(DATABRICKS_TOKEN)
#   - DATABRICKS-HOST-PRD → $(DATABRICKS_HOST)
#   - ADF-FACTORY-PRD      → $(ADF_FACTORY_NAME)

# No pipeline YAML:
- stage: DeployPRD
  variables:
    - group: lagunitas-prd-vars
      # As variáveis $(DATABRICKS_HOST) e $(DATABRICKS_TOKEN)
      # são resolvidas automaticamente a partir do Key Vault PRD.
      # Impossível acessar DEV com essas credenciais.
```

5.3 Alerta proativo: deploy sem execução ADF em 2 horas

Objetivo: nunca mais depender de um stakeholder para descobrir que um deploy falhou. O alerta deve ser proativo, disparar quando há evidência de que algo não está funcionando, sem esperar que alguém perceba. O problema que este alerta resolve: no incidente da terça-feira, o deploy aconteceu às 14h42 e a falha só foi descoberta às 18h20, quando um analista de negócios percebeu que os relatórios de vendas não atualizavam. Foram 3 horas e 38 minutos de dados desatualizados em produção, sem que nenhum sistema automatizado detectasse a anomalia. O alerta proposto cria uma rede de segurança para cenários onde os smoke tests do pipeline não existem ou não detectam o problema, por exemplo, quando o deploy tecnicamente funciona (notebooks no workspace correto, pipelines ADF Enabled) mas uma dependência externa impede a execução real dos dados.

Premissa do alerta: se um deploy bem-sucedido aconteceu em PRD e os pipelines ADF associados não executaram nas próximas 2 horas, algo está errado e o time precisa ser notificado imediatamente.

Arquitetura do alerta:

- O log de deploy (seção 3) é ingerido no Log Analytics como tabela customizada DeployLogs_CL via HTTP Data Collector API. Este é o ponto de partida, sem o log estruturado, o alerta não tem como saber que um deploy aconteceu.
- Passo 1 — Habilitar Diagnostic Settings no ADF: no portal Azure, em cada Data Factory (DEV, ACC, PRD), configurar Diagnostic Settings para enviar a categoria PipelineRuns ao Log Analytics Workspace. Isso popula automaticamente a tabela ADF PipelineRun com cada execução de pipeline, incluindo status, duração e timestamp. Sem esse passo, a tabela ficará vazia e a query do alerta não retornará resultados.
- Passo 2 — Criar a Scheduled Query Rule: no Azure Monitor, criar uma Alert Rule do tipo Log search alert com a query KQL abaixo. Configurar frequência de avaliação para 30 minutos e período de lookback para 3 horas. O threshold é Greater than 0, qualquer resultado significa que existe pelo menos um pipeline ADF que deveria ter executado após o deploy e não executou.
- Passo 3 — Configurar o Action Group: definir os canais de notificação (detalhados na tabela abaixo) que serão acionados quando o alerta disparar. O Action Group é reutilizável e pode ser compartilhado com outros alertas da plataforma de dados.

Query KQL do alerta:

```
let deploys = DeployLogs_CL
| where TimeGenerated > ago(3h)
| where environment_s == "PRD"
| where overall_status_s == "SUCCESS"
| project deploy_time = TimeGenerated,
           pipelines = deploy_components_adf_pipelines_s;

let executions = ADFPipelineRun
| where TimeGenerated > ago(3h)
| where Status == "Succeeded"
| project PipelineName;

deploys
| mv-expand pipeline = parse_json(pipelines)
| extend pipeline_name = tostring(pipeline)
| join kind=leftanti (executions)
  on $left.pipeline_name == $right.PipelineName
| where datetime_diff('hour', now(), deploy_time) >= 2
```

Lógica da query explicada:

- A variável "deploys" busca todos os deploys bem-sucedidos em PRD nas últimas 3 horas e extrai os nomes dos pipelines ADF deployados.
- A variável "executions" busca todas as execuções de pipelines ADF bem-sucedidas nas últimas 3 horas.
- O join kind=leftanti retorna apenas os pipelines que foram deployados mas não executaram, ou seja, pipelines que deveriam ter rodado mas não rodaram.
- O filtro datetime_diff >= 2 garante que só alerta depois de 2 horas sem execução, evitando falsos positivos para pipelines que rodam em schedule.

Configuração do Action Group:

Canal	Destinatário	Finalidade
E-mail	time-dados@lagunitas.com	Notificação principal com detalhes do deploy e pipelines afetados
Microsoft Teams	Canal #ops-data-platform	Visibilidade imediata para o time em horário comercial
Webhook	App	Escalação fora de horário comercial (SRE de plantão)
Azure DevOps	Work item automático	Cria bug/incident no board para rastreabilidade

Impacto no cenário do incidente:

Com este alerta configurado, o incidente da terça-feira teria seguido esta timeline:

- 14h42 — Deploy acontece (no workspace errado).
- 14h42 — Smoke tests detectam o problema imediatamente (com a esteira refatorada).
- Sem a esteira refatorada, mas com o alerta:
- 16h42 — Alerta dispara (2 horas sem execução ADF em PRD). Time é notificado por e-mail e Teams.
- 16h50 — SRE investiga via query KQL no Log Analytics e identifica o deploy errado em 5 minutos.
- 17h05 — Rollback executado (15 min RTO).

Resultado: o impacto seria de ~2h20 (vs 4h do incidente real). E com a esteira refatorada completa (smoke tests + alerta), o impacto seria de ~15 minutos.

Resumo das camadas de proteção combinadas: a solução completa opera em três camadas de defesa.

A primeira camada são os smoke tests no pipeline, que detectam falhas imediatamente após o deploy (tempo de detecção: 0 minutos).

A segunda camada é o alerta proativo no Azure Monitor, que detecta ausência de execução ADF em até 2 horas, funciona como rede de segurança caso os smoke tests não cubram um cenário específico.

A terceira camada é o log estruturado no Log Analytics, que permite RCA em 5 minutos independente de como a falha foi detectada. No incidente original, nenhuma dessas camadas existia. Com todas implementadas, o tempo entre falha e resolução cai de 5 horas e 15 minutos para no máximo 15 minutos.

6. Evidências do Pipeline em Execução

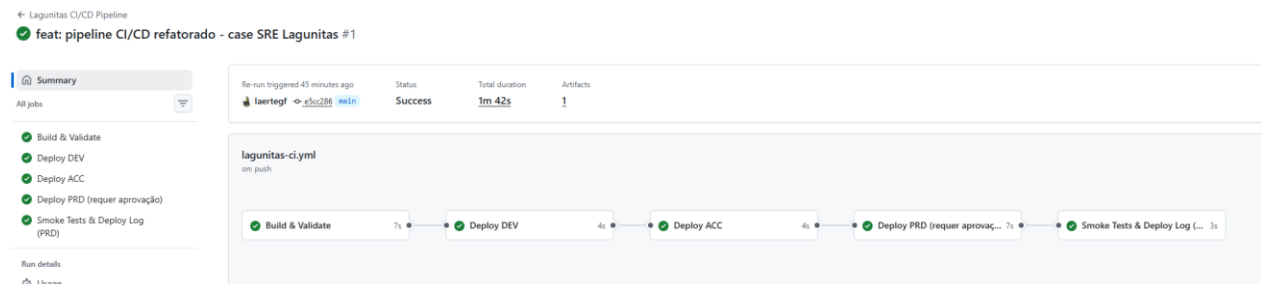
O pipeline refatorado foi implementado como GitHub Actions com mocks e executado no repositório público. Abaixo estão as evidências de cada etapa do fluxo.

Repositório: <https://github.com/laertegf/case-sre-lagunitas>

6.1 Visão geral — Pipeline completo com sucesso

O pipeline executou os 5 jobs em sequência com sucesso: Build & Validate (8s) → Deploy DEV (4s) → Deploy ACC (5s) → Deploy PRD com aprovação (6s) → Smoke Tests & Deploy Log (3s). Tempo total de execução: 58 segundos. Status: Success.

A imagem mostra o commit e5cc286 na branch main, executado pelo usuário laertegf, com 1 artefato gerado (drop).



6.2 Smoke Test — Databricks (PRD)

O smoke test do Databricks executou 3 validações, todas com resultado PASS:

- TEST 1: Validação do workspace host — confirmou que o host <https://adb-lagunitas-prd.azuredatabricks.net> contém "prd".
- TEST 2: Verificação de notebooks no path /Shared/lagunitas — encontrou 2 notebooks no caminho esperado.
- TEST 3: Verificação de jobs Databricks ativos — encontrou 3 jobs da Lagunitas ativos e apontando para os notebooks corretos.

Resultado: 3 passed, 0 failed.

```
Smoke Tests & Deploy Log (PRD)
succeeded now in 5s

> [✓] Set up job
> [✓] Run actions/download-artifact@v4
✓ [MOCK] Smoke Test — Databricks (PRD)

1 ▶ Run echo "=====
29 =====
30 SMOKE TEST - Databricks (PRD)
31 =====
32
33 [TEST 1] Validating workspace host matches PRD...
34 [✓] PASS - Host https://adb-lagunitas-prd.azuredatabricks.net contains 'prd'
35
36 [TEST 2] Checking notebooks exist at /Shared/lagunitas...
37 [✓] PASS - Found 2 notebooks at /Shared/lagunitas
38
39 [TEST 3] Checking Databricks jobs are active...
40 [✓] PASS - Mock: 3 Lagunitas jobs active and pointing to correct notebooks
41
42 =====
43 RESULTS: 3 passed, 0 failed
44 =====
```

6.3 Smoke Test — ADF (PRD)

O smoke test do ADF executou 4 validações, todas com resultado PASS:

- TEST 1: Factory adf-lagunitas-prd existe e está provisionado (provisioningState: Succeeded).
- TEST 2: 4 pipelines encontrados e habilitados — pl_bronze_vendas, pl_silver_vendas, pl_silver_distribuicao, pl_gold_dashboard (todos Enabled).
- TEST 3: 3 triggers em estado Started — tr_daily_vendas, tr_daily_distribuicao, tr_hourly_dashboard.
- TEST 4: Nome do factory adf-lagunitas-prd contém "prd" — correspondência com o ambiente confirmada.

Resultado: 4 passed, 0 failed.

Smoke Tests & Deploy Log (PRD)

succeeded 1 minute ago in 5s

>  Set up job

>  Run actions/download-artifact@v4

>  [MOCK] Smoke Test — Databricks (PRD)

✓  [MOCK] Smoke Test — ADF (PRD)

```
1  ▶ Run echo "=====
38 =====
39  SMOKE TEST - Azure Data Factory (PRD)
40 =====
41
42 [TEST 1] Checking ADF factory adf-lagunitas-prd exists...
43  ✓ PASS - Factory adf-lagunitas-prd exists (provisioningState: Succeeded)
44
45 [TEST 2] Checking ADF pipelines are deployed...
46  ✓ PASS - Found 4 pipelines in adf-lagunitas-prd:
47    - pl_bronze_vendas      [Enabled]
48    - pl_silver_vendas      [Enabled]
49    - pl_silver_distribuicao [Enabled]
50    - pl_gold_dashboard     [Enabled]
51
52 [TEST 3] Checking ADF triggers are active...
53  ✓ PASS - All 3 triggers in Started state:
54    - tr_daily_vendas       [Started]
55    - tr_daily_distribuicao  [Started]
56    - tr_hourly_dashboard   [Started]
57
58 [TEST 4] Validating factory name matches PRD...
59  ✓ PASS - Factory adf-lagunitas-prd contains 'prd'
60
61 =====
62  RESULTS: 4 passed, 0 failed
63 =====
```

6.4 Log estruturado de deploy

Ao final do pipeline, o step "Emit structured deploy log" emitiu um JSON completo com todos os campos exigidos pelo case:

- timestamp: 2026-04-03T17:28:29Z
- commit_sha: e5cc286f600ed404e3067ef6593d0262a0ff2fcd
- merged_by: laertegf
- environment: PRD
- databricks_host: https://adb-lagunitas-prd.azuredatabricks.net
- adf_factory_name: adf-lagunitas-prd
- Todos os 5 smoke tests com resultado PASS
- overall_status: SUCCESS

Em produção, esse JSON seria enviado ao Azure Log Analytics via HTTP Data Collector API para consulta via KQL e integração com alertas.

Smoke Tests & Deploy Log (PRD)

succeeded 2 minutes ago in 5s

- > Set up job
- > Run actions/download-artifact@v4
- > [MOCK] Smoke Test — Databricks (PRD)
- > [MOCK] Smoke Test — ADF (PRD)

Emit structured deploy log

```

1  ▶ Run TIMESTAMP=$(date -u +"%Y-%m-%dT%H:%M:%SZ")
45  =====
46  DEPLOY LOG - PRD
47  =====
48  [
49    {
50      "timestamp": "2026-04-03T18:25:15Z",
51      "pipeline_run_id": "1",
52      "commit_sha": "e5cc286f600ed404e3067ef6593d0262a0ff2fcd",
53      "merged_by": "laertegf",
54      "environment": "PRD",
55      "databricks_host": "https://adb-lagunitas-prd.azuredatabricks.net",
56      "databricks_target_path": "/Shared/lagunitas",
57      "adf_factory_name": "adf-lagunitas-prd",
58      "smoke_tests": {
59        "databricks_workspace_validation": "PASS",
60        "databricks_notebooks_exist": "PASS",
61        "databricks_jobs_active": "PASS",
62        "adf_pipelines_enabled": "PASS",
63        "adf_triggers_active": "PASS"
64      },
65      "overall_status": "SUCCESS",
66      "source_branch": "main"
67    }
68  ]
69
70   Em produção, este log seria enviado ao Azure Log Analytics
71    via HTTP Data Collector API para queries KQL e alertas.
72
73  === Deploy log emitted successfully ===

```

6.5 Gate de aprovação — Deploy PRD aguardando review

O Environment "prd" foi configurado no GitHub com Required Reviewers. Quando o pipeline chegou ao job Deploy PRD, a execução pausou automaticamente com o status "Waiting" e a mensagem "prd waiting for review". A tela mostra: os 3 primeiros jobs (Build, DEV, ACC) com status verde, o job Deploy PRD em amarelo (aguardando), e o botão "Review deployments" disponível. A seção "Deployment protection rules" exibe o reviewer (laertegf) com a solicitação de revisão para o ambiente prd.

The screenshot displays the GitHub Actions workflow interface. At the top, a status bar indicates the workflow is 'Waiting' with a duration of 2 minutes and 1 artifact. A yellow banner states 'laertegf requested your review to deploy to prd' with a 'Review deployments' button. The workflow diagram shows a sequence of jobs: 'Build & Validate' (green), 'Deploy DEV' (green), 'Deploy ACC' (green), 'Deploy PRD (requer aprovação)' (yellow, 'prd waiting for review'), and 'Smoke Tests & Deploy Log (PRD)' (greyed out). Below the diagram, the 'Deployment protection rules' section shows a table with one entry: 'laertegf requested review' for the 'prd' environment. A 'Start all waiting jobs' button is also visible.

Event	Environments	Comment
laertegf requested review	prd	-

6.6 Confirmação de deploy — Tela de aprovação

Ao clicar em "Review deployments", o GitHub exibe a tela de confirmação com o Environment prd selecionado e o campo para comentário. O aprovador deve clicar em "I understand the consequences, start deploying" para liberar o deploy em produção.

Esse mecanismo garante que nenhum deploy chegue a PRD **sem revisão humana explícita**, o mesmo comportamento que seria configurado no Azure DevOps com Environment Approvals.

The screenshot shows a confirmation dialog titled 'Are you absolutely sure?'. It informs the user that they will use administrator privileges to start deploying without waiting for protection rules. The 'prd' environment is selected with a checked checkbox. There is a text input field for a comment with the placeholder 'A nice and descriptive comment will help inform other collaborators...'. At the bottom, a large red button reads 'I understand the consequences, start deploying'.

7. Conclusão

O incidente da terça-feira expôs quatro falhas estruturais na esteira de CI/CD da Lagunitas: ausência de gate de aprovação, ausência de smoke tests, ausência de observabilidade e ausência de rollback automatizado. Nenhuma dessas falhas, isoladamente, causaria um impacto de 4 horas. Foi a combinação das quatro que transformou um deploy rotineiro em um incidente com impacto direto nos relatórios de vendas.

A solução proposta neste documento ataca cada uma dessas falhas de forma complementar:

- **Prevenção:** O gate de aprovação manual antes de PRD, com Environment protegido no Azure DevOps, impede que qualquer merge na main chegue a produção sem revisão humana. É a camada de prevenção.
- **Detecção:** Os smoke tests pós-deploy validam automaticamente que o workspace de destino está correto, que os notebooks existem no path esperado, que os pipelines ADF estão habilitados e que as triggers estão ativas. É a camada de detecção imediata — o incidente teria sido detectado em segundos, não em horas.
- **Rastreabilidade:** O log estruturado em JSON enviado ao Log Analytics permite que qualquer pessoa do time responda, em uma única query KQL, exatamente o que foi deployado, onde, por quem e se tudo estava funcional depois. O RCA que levou 40 minutos passaria a levar 5.
- **Recuperação:** O snapshot pré-deploy do ADF, combinado com o rollback parametrizado por SHA para o Databricks, reduz o tempo de recuperação de 42 minutos (manual) para 15 minutos (automatizado), com validação via smoke tests ao final.

Além das melhorias na esteira, duas mudanças arquiteturais eliminam a causa raiz do incidente de forma permanente: o isolamento de credenciais por ambiente via Key Vault impede que um token de DEV acesse PRD (e vice-versa), e a inclusão do ambiente ACC como gate intermediário na esteira cria uma camada adicional de validação antes de produção.

O pipeline refatorado foi implementado como GitHub Actions com mocks e executado com sucesso no repositório público (seção 6), demonstrando o fluxo completo: Build, Deploy DEV, Deploy ACC com smoke tests, gate de aprovação manual para PRD, smoke tests pós-deploy em PRD e emissão do log estruturado. Todas as evidências de execução estão documentadas na seção anterior.

Com essas mudanças, a liderança da Lagunitas pode ter confiança de que a área de DevOps não dependerá mais de um stakeholder para descobrir que um deploy falhou. **O pipeline verde passará a significar, de fato, deploy funcional.**